

Solution Requirements

Date	4 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	The system currently operates without user login (public-facing tool). Users access the loan prediction form directly via the web browser without needing to create an account or verify identity. Future versions may add bank-officer login via username/password with session management.	Low
2	Authorization Levels	Single public role: Any user (loan applicant) can access the form and receive a prediction. No admin panel exists in the current version. A future admin role could access prediction logs and model performance stats.	Low
3	External Interfaces	The system uses Google Fonts API (fonts.googleapis.com) for UI typography. The ML model is loaded locally from model.pkl (no external ML API). The dataset dataset/loan.csv is stored locally. No third-party payment or banking API is integrated.	Medium
4	Transaction Processing	When a user submits the loan application form (POST /predict), the system: (1) reads and encodes 11 input fields, (2) passes the feature array to the XGBoost model, (3) returns an Approved or Rejected prediction along with an applicant summary — all within a single synchronous HTTP request-response cycle.	High
5	Reporting	The result page displays an applicant summary table showing all 11 submitted fields with their values (formatted, e.g., income in ■). It also displays key decision factors (Credit History, Income Level, Property Area, Education) explaining the model's decision. No CSV/PDF export or dashboard is available in the current version.	Medium

S.No	Requirement Category	Requirement Description	Priority
6	Business Rules	(1) Credit history (Good/Bad) is the strongest predictor of approval. (2) Loan amount is stored in thousands (e.g., 150 = Rs. 1,50,000). (3) Dependents of 3 or more are grouped as "3+". (4) The XGBoost model is selected only if it achieves the highest test accuracy among Decision Tree, Random Forest, KNN, and XGBoost. (5) The result page shows a "Review & Resubmit" button only on rejection.	High
7	Compliance to Laws or Regulations	The system is for educational purposes only; the result page explicitly states "Final loan decisions are made by the bank." No real financial data is stored or transmitted. No personal user data is persisted after the session. Should comply with basic data minimization principles (GDPR-aligned) — applicant data is not stored server-side.	Medium
8	Other	(1) The system must handle form submission errors gracefully and display a user-friendly error message on the result page. (2) The model training script (train_model.py) must handle missing values via mode/mean imputation before training. (3) The system supports re-submission — users can click "Apply Again" to restart.	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The loan prediction result must be returned quickly after form submission. The XGBoost model inference is lightweight (11 features), so response time should be near-instant. Page load should be fast with minimal assets.	Prediction result delivered in < 2 seconds; Page load in < 3 seconds on a standard broadband connection
2	Scalability	The Flask development server handles single-threaded requests. For production, it should be deployed behind a WSGI server (e.g., Gunicorn) to handle concurrent users. The model is stateless and loaded once at startup, making horizontal scaling straightforward.	Supports at least 50 concurrent users in production deployment with Gunicorn; model loaded once at startup
3	Security & Data Privacy	Applicant data submitted via the form is not persisted to any database or log file. The application does not expose model internals. Input validation is applied on all 11 form fields (required fields, numeric ranges). Debug mode must be disabled in production.	No applicant PII stored server-side; all form fields validated before model inference; debug=False in production
4	Reliability & Availability	The application must gracefully handle unexpected inputs or model errors via the try/except block in /predict, returning a friendly error message instead of crashing. The pre-trained model.pkl must be available at startup.	99% uptime during usage sessions; zero unhandled crashes — all exceptions caught and displayed as friendly errors
5	Usability & Accessibility	The UI uses a clean, responsive design (Inter font, grid layout, mobile viewport meta tag). All form fields have labels, placeholders, and required validation. Result page clearly distinguishes Approved (green) and Rejected (red) states. The "How It Works" section guides first-time users.	Form usable on desktop and mobile screens; all inputs labeled; color is not the only visual indicator (icons + text used); complies with basic WCAG 2.1 AA principles

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
6	Other (Maintainability & Portability)	The codebase is modular — training (train_model.py) is separate from serving (app.py). All dependencies are pinned in requirements.txt. The model can be retrained by simply running train_model.py and replacing model.pkl. The app runs on any OS with Python 3.x installed.	New model deployable by replacing model.pkl without code changes; all dependencies installable via pip install -r requirements.txt; runs on Windows, Linux, and macOS